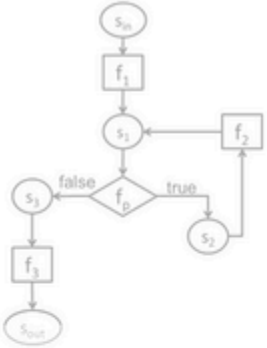
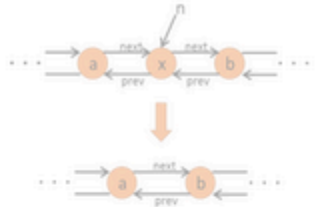
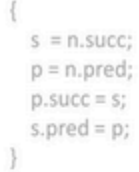
$$\exists c \forall in\ Q(c, in)$$

```
/* Average of x and y without using x+y (avoid overflow)*/
int avg(int x, int y){
  int t = expr({x/2, y/2, x%2, y%2, 2 }, {PLUS, DIV});
  assert t == (x+y)/2;
  return t;
}
```

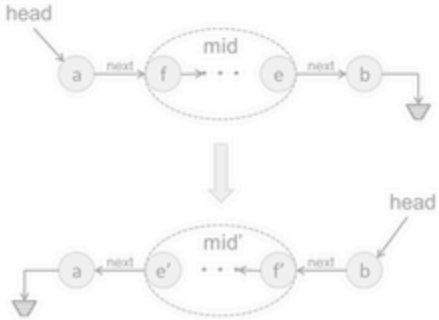


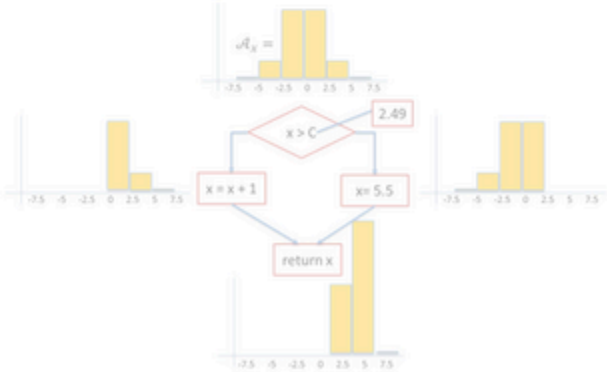




```
{
  s = n.succ;
  p = n.pred;
  p.succ = s;
  s.pred = p;
}
```

Introduction to Principles and Practice of Software Synthesis





$$\varphi(p)$$

$$Sk[c](in)$$

Lecture 1
Course Overview and Introduction to Synthesis

Armando Solar-Lezama

The promise of automation

The FORTRAN Automatic Coding System

J. W. BACKUS†, R. J. BEEBER†, S. BEST‡, R. GOLDBERG†, L. M. HAIBT†,
H. L. HERRICK†, R. A. NELSON†, D. SAYRE†, P. B. SHERIDAN†,
H. STERN†, I. ZILLER†, R. A. HUGHES§, AND R. NUTT‖

INTRODUCTION

THE FORTRAN project was begun in the summer of 1954. Its purpose was to reduce by a large factor the task of preparing scientific problems for IBM's next large computer, the 704. If it were possible for the 704 to code problems for itself and produce as good programs as human coders (but without the errors), it was clear that large benefits could be achieved. For it was known that about two-thirds of the cost of solving most scientific and engineering problems on large computers was that of problem preparation. Furthermore, more than 90 per cent of the elapsed time for a problem was usually devoted to planning, writing,

system is now complete. It has two components: the FORTRAN language, in which programs are written, and the translator or executive routine for the 704 which effects the translation of FORTRAN language programs into 704 programs. Descriptions of the FORTRAN language and the translator form the principal sections of this paper.

The experience of the FORTRAN group in using the system has confirmed the original expectations concerning reduction of the task of problem preparation and the efficiency of output programs. A brief case history of one job done with a system seldom gives a good measure of its usefulness, particularly when the

The promise of automation

The FORTRAN Automatic Coding System

J. W. BACKUS†, R. J. BEEBER†, S. BEST‡, R. GOLDBERG†, L. M. HAIBT†,
H. L. HERRICK†, R. A. NELSON†, D. SAYRE†, P. B. SHERIDAN†,
H. STERN†, I. ZILLER†, R. A. HUGHES§, AND R. NUTT‖

IBM's next large computer, the 704. If it were possible for the 704 to code problems for itself and produce as good programs as human coders (but without the errors), it was clear that large benefits could be achieved.

for a problem was usually devoted to planning, writing, good measure of its usefulness, particularly when the

What is software synthesis?

IEEE TRANSACTIONS ON SOFTWARE ENGINEERING, VOL. SE-5, NO. 4, JULY 1979

Synthesis: Dreams $\Longrightarrow$ Programs

ZOHAR MANNA AND RICHARD WALDINGER

: techniques are presented for deriving programs
iven specifications. The specifications express the
d program without giving any hint of the algo-
l. The basic approach is to transform the specifi-
cording to certain rules, until a satisfactory pro-

INTRODUCTION

IN RECENT years there has been increasing activity in the field of program verification. The goal of these efforts is to construct computer systems for determining whether a



Zohar Manna



Richard Waldinger

What is software synthesis?

When someone says "I want a programming language in which I need only say what I wish done," give him a lollipop.

-Allan Perlis 1982



The synthesis conundrum



I don't want to program

- The machine should program for me

But I need to tell the machine what I want

- I need a notation to describe what I want
- with great precision with little room for ambiguity

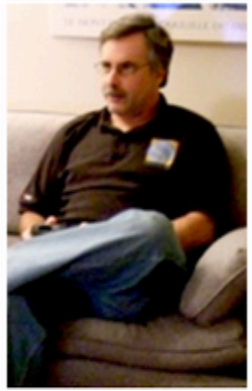
So instead of "programming"

- I will write detailed step by step descriptions of system behavior
- In a notation that requires great mathematical sophistication
- That I have never used before (unlike my favorite programming language which I started using in grade school)

What is software synthesis?

Programmer's Apprentice (MIT 1987)

- Codify expert knowledge on how to solve programming problems
- User guided synthesis
- Collaborative model is strictly additive
  - Programmer is not prevented from just writing code, but gets assistance when required



Charles Rich



Richard C. Waters

Software synthesis checklist

Are you generating software?

How do you express what you want the software to do?

- what => how

How sensitive is the system to how you write the spec?

- C compiler?

How much is the system doing for you at compile-time?

- Declarative programming, e.g., Prolog?

Structure of the Course

Unit 1: Inductive Synthesis

Unit 2: Functional Synthesis

Unit 3: Quantitative Synthesis and Learning

Themes for the course

How do you tell the system what you want?

- What is the specification formalism
- What is the interaction model
- What happens when there is ambiguity

How do you represent domain knowledge?

- How do you guide the system towards relevant programs
- How can you take advantage of the structure of the space of programs you care about

How does the system find the program you actually want?

- And how does it know it is the program you want

Unit 1: Inductive Synthesis

Synthesize a program whose behavior satisfies a set of examples

Doesn't machine learning do that?

Traditional Machine Learning

- Learn a function from a set of examples
- Scalability is very important, algorithms must scale to millions of data points
- Data is assumed to be noisy;
  - need to avoid overfitting
- Space of possible functions is highly stylized
- Background knowledge incorporated as preprocessing and feature selection

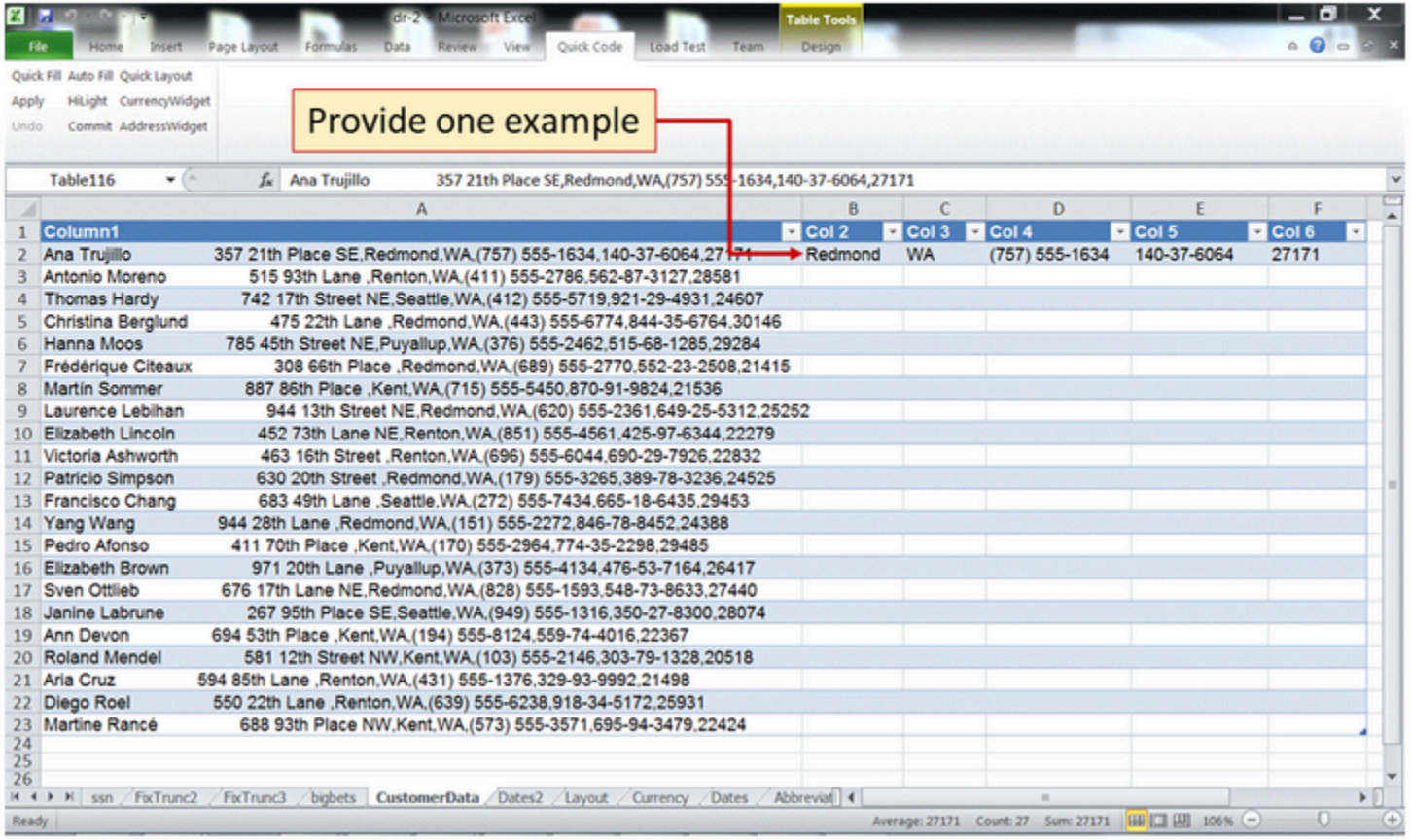
Inductive Synthesis

- Learn a function from a set of examples
- Scalability is not so important, usually we are dealing with small numbers of examples
- Data is assumed to be clean
  - It's annoying when user says f(x)=y and the system assumes the user is wrong and decides that f(x)=z
- Space of possible functions can be arbitrary
- Background knowledge encoded in the description of the space and in the search itself

Real world application of synthesis



FlashFill: a feature of Excel 2013 (Sumit Gulwani et al.)



FlashFill: a feature of Excel 2013 (Sumit Gulwani et al.)

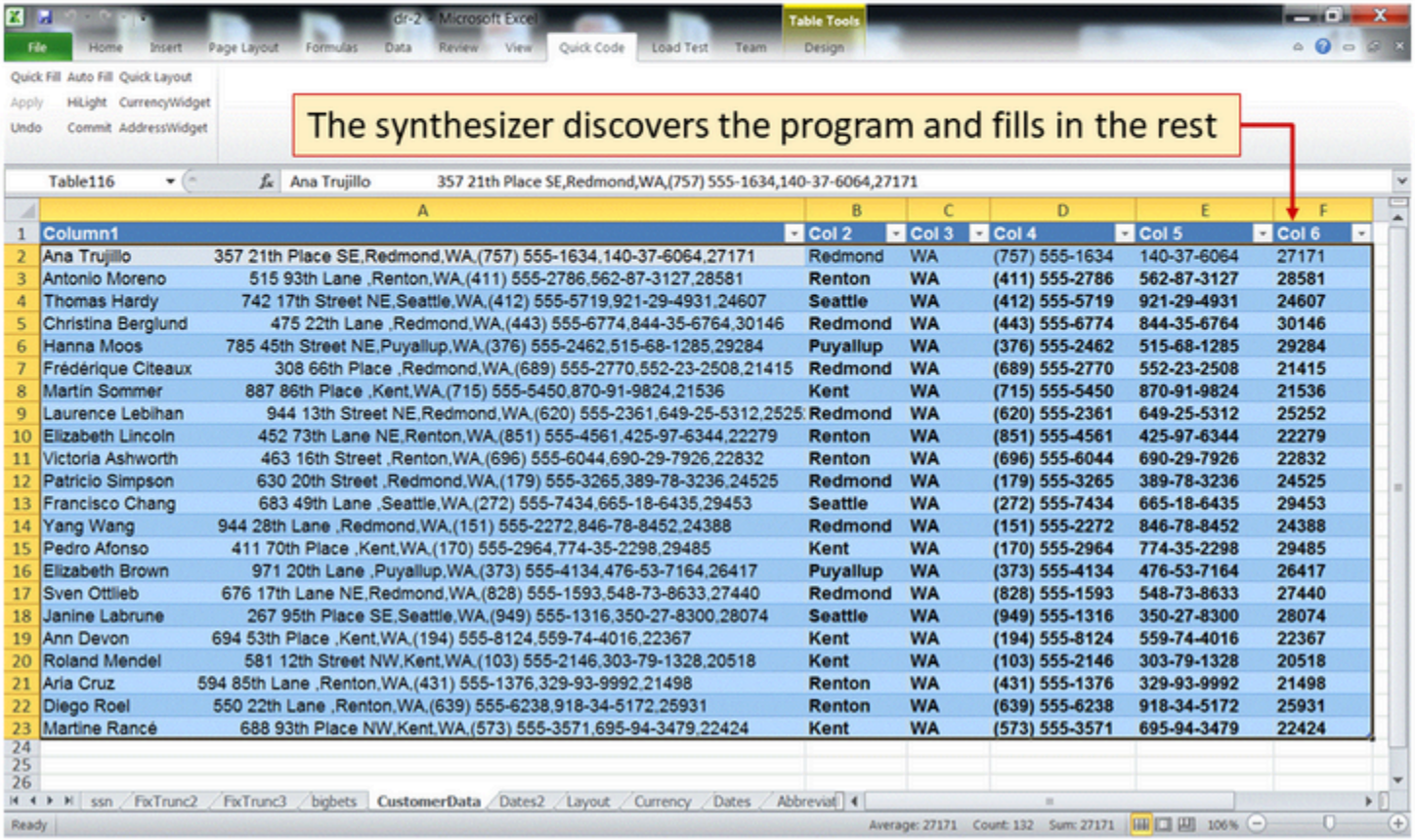


Table116 | Ana Trujillo | 357 21th Place SE,Redmond,WA,(757) 555-1634,140-37-6064,27171

	A	B	C	D	E	F
1	Column1	Col 2	Col 3	Col 4	Col 5	Col 6
2	Ana Trujillo 357 21th Place SE,Redmond,WA,(757) 555-1634,140-37-6064,27171	Redmond	WA	(757) 555-1634	140-37-6064	27171
3	Antonio Moreno 515 93th Lane ,Renton,WA,(411) 555-2786,562-87-3127,28581	Renton	WA	(411) 555-2786	562-87-3127	28581
4	Thomas Hardy 742 17th Street NE,Seattle,WA,(412) 555-5719,921-29-4931,24607	Seattle	WA	(412) 555-5719	921-29-4931	24607
5	Christina Berglund 475 22th Lane ,Redmond,WA,(443) 555-6774,844-35-6764,30146	Redmond	WA	(443) 555-6774	844-35-6764	30146
6	Hanna Moos 785 45th Street NE,Puyallup,WA,(376) 555-2462,515-68-1285,29284	Puyallup	WA	(376) 555-2462	515-68-1285	29284
7	Frédérique Citeaux 308 66th Place ,Redmond,WA,(689) 555-2770,552-23-2508,21415	Redmond	WA	(689) 555-2770	552-23-2508	21415
8	Martin Sommer 887 86th Place ,Kent,WA,(715) 555-5450,870-91-9824,21536	Kent	WA	(715) 555-5450	870-91-9824	21536
9	Laurence Lebihan 944 13th Street NE,Redmond,WA,(620) 555-2361,649-25-5312,2525	Redmond	WA	(620) 555-2361	649-25-5312	25252
10	Elizabeth Lincoln 452 73th Lane NE,Renton,WA,(851) 555-4561,425-97-6344,22279	Renton	WA	(851) 555-4561	425-97-6344	22279
11	Victoria Ashworth 463 16th Street ,Renton,WA,(696) 555-6044,690-29-7926,22832	Renton	WA	(696) 555-6044	690-29-7926	22832
12	Patricio Simpson 630 20th Street ,Redmond,WA,(179) 555-3265,389-78-3236,24525	Redmond	WA	(179) 555-3265	389-78-3236	24525
13	Francisco Chang 683 49th Lane ,Seattle,WA,(272) 555-7434,665-18-6435,29453	Seattle	WA	(272) 555-7434	665-18-6435	29453
14	Yang Wang 944 28th Lane ,Redmond,WA,(151) 555-2272,846-78-8452,24388	Redmond	WA	(151) 555-2272	846-78-8452	24388
15	Pedro Afonso 411 70th Place ,Kent,WA,(170) 555-2964,774-35-2298,29485	Kent	WA	(170) 555-2964	774-35-2298	29485
16	Elizabeth Brown 971 20th Lane ,Puyallup,WA,(373) 555-4134,476-53-7164,26417	Puyallup	WA	(373) 555-4134	476-53-7164	26417
17	Sven Ottlieb 676 17th Lane NE,Redmond,WA,(828) 555-1593,548-73-8633,27440	Redmond	WA	(828) 555-1593	548-73-8633	27440
18	Janine Labrune 267 95th Place SE,Seattle,WA,(949) 555-1316,350-27-8300,28074	Seattle	WA	(949) 555-1316	350-27-8300	28074
19	Ann Devon 694 53th Place ,Kent,WA,(194) 555-8124,559-74-4016,22367	Kent	WA	(194) 555-8124	559-74-4016	22367
20	Roland Mendel 581 12th Street NW,Kent,WA,(103) 555-2146,303-79-1328,20518	Kent	WA	(103) 555-2146	303-79-1328	20518
21	Aria Cruz 594 85th Lane ,Renton,WA,(431) 555-1376,329-93-9992,21498	Renton	WA	(431) 555-1376	329-93-9992	21498
22	Diego Roel 550 22th Lane ,Renton,WA,(639) 555-6238,918-34-5172,25931	Renton	WA	(639) 555-6238	918-34-5172	25931
23	Martine Rancé 688 93th Place NW,Kent,WA,(573) 555-3571,695-94-3479,22424	Kent	WA	(573) 555-3571	695-94-3479	22424
24						
25						
26						

ssn | FixTrunc2 | FixTrunc3 | bigbets | CustomerData | Dates2 | Layout | Currency | Dates | Abbreviat

Ready | Average: 27171 | Count: 132 | Sum: 27171 | 106%

FlashFill Demo

Unit 2: Functional Synthesis

Goal: Synthesize a function that satisfies a specification

- How do we know the specification has been satisfied? Isn't verification itself already quite hard?
- Can we leverage inductive synthesis machinery for this problem?
- What is the relevant space of functions?
- How do we explore this space efficiently?

Example: Least Significant Zero Bit

- 0010 0101 → 0000 0010

```
int W = 32;
bit[W] isolate0 (bit[W] x) {        // W: word size
       bit[W] ret = 0;
       for (int i = 0; i < W; i++)
              if (!x[i]) { ret[i] = 1; return ret;   }
}
```

Trick:

- Adding 1 to a string of ones turns the next zero to a 1
- i.e. 000111 + 1 = 001000

Space of possible implementations

```
/**
 * Generate the set of all bit-vector expressions
 * involving +, &, xor and bitwise negation (~).
 * the bnd param limits the size of the generated expression.
 */

generator bit[W] gen(bit[W] x, int bnd){
    assert bnd > 0;
    if(??) return x;
    if(??) return ??;
    if(??) return ~gen(x, bnd-1);
    if(??){
        return {| gen(x, bnd-1) (+ | & | ^) gen(x, bnd-1) |};
    }
}
```

Example: Least Significant Zero Bit

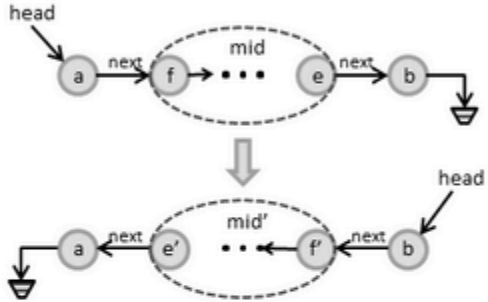
```
generator bit[W] gen(bit[W] x, int bnd){
    assert bnd > 0;
    if(??) return x;
    if(??) return ??;
    if(??) return ~gen(x, bnd-1);
    if(??){
        return {| gen(x, bnd-1) (+ | & | ^) gen(x, bnd-1) |};
    }
}

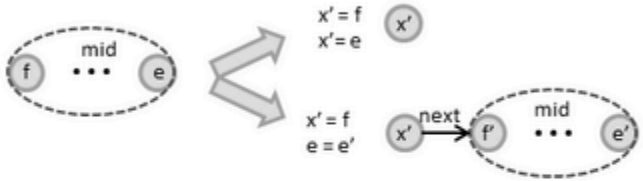
bit[W] isolate0sk (bit[W] x)  implements isolate0 {
     return gen(x, 3);
}
```

Sketch Demo

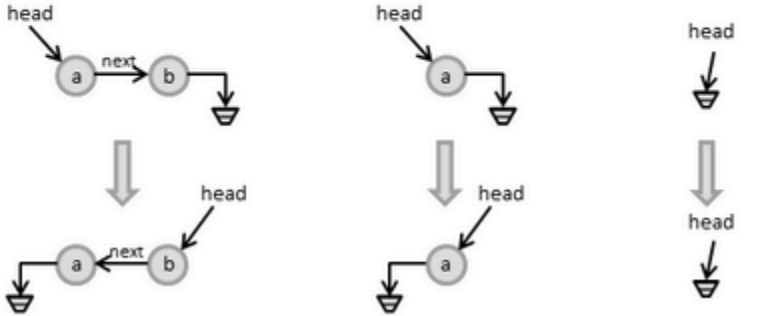
Storyboard Programming

Led by Rishabh Singh





Abstract Scenarios



Concrete Scenarios

```
void llReverse(Node head)
    {
        ?? /*1*/
        while (?? /*p*/)
        {
            ?? /*2*/
        }
        ?? /*3*/
    }
```

Structural Info

Storyboard Programming

```
void llReverse(Node head)
    {
        Node temp1 = null, temp2 = null,
        temp3 = null;
        temp1 = head;
        while (temp1 != null)
        {
            // unfold temp1;
            head = temp1;
            temp1 = temp1.next;
            head.next = head;
            head.next = temp3;
            temp3 = head;
            // fold temp3;
        }
    }
```

Unit 3: Quantitative Synthesis and Learning

Boolean specifications are often insufficient

- They can be too hard to write
- Fail to capture important properties of the problem

A Bayesian view of synthesis

- Probabilities reflect our belief about which program is wanted
- Priors over programs are updated with new evidence
- Need new mechanisms to capture distributions over programs
- Search must be guided by these probabilities

Applications

Reverse engineering

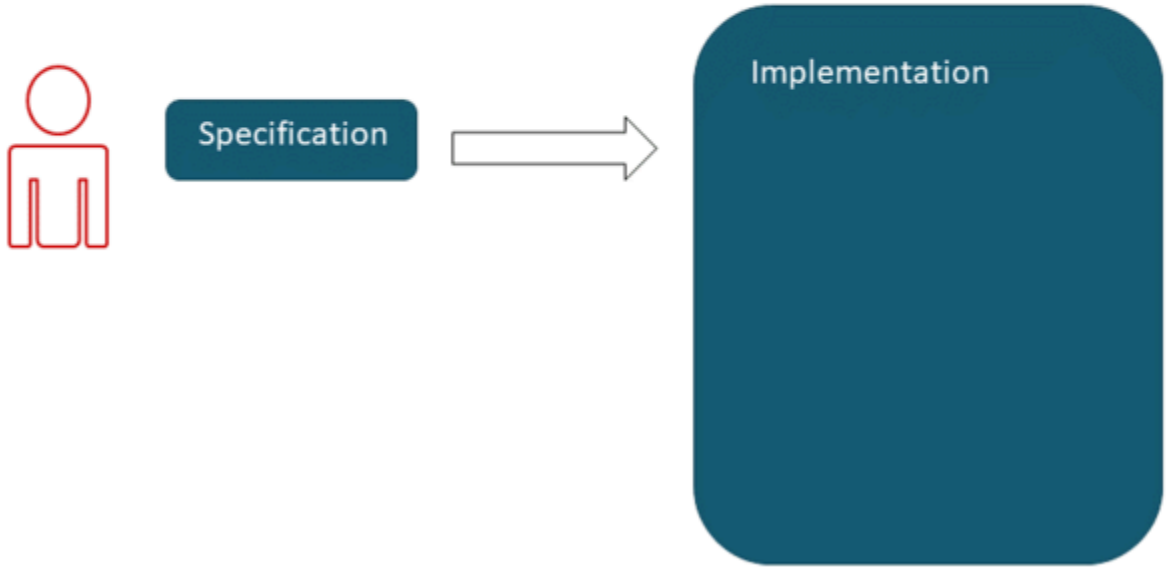
Oracle-guided component-based program synthesis

- ICSE 2010 paper by Jha, Gulwani, Seshia and Tiwari

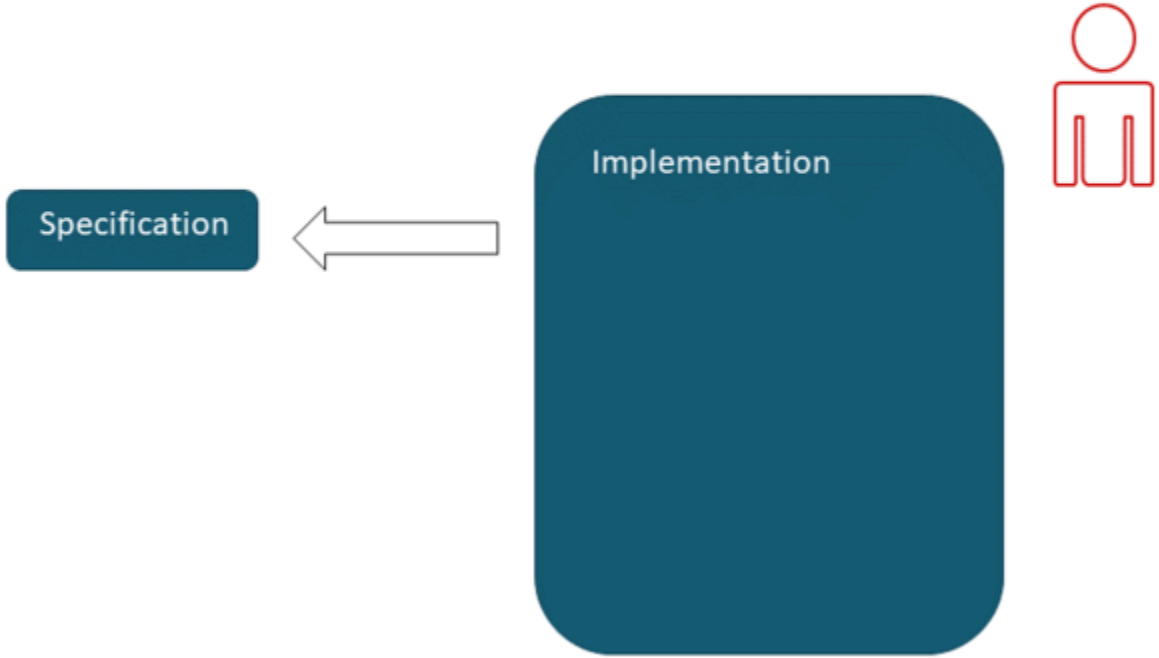
Pioneered a number of new ideas at the algorithmic level

Synthesis for reverse engineering

Reverse engineering



Reverse engineering



Verified Lifting

Synthesis based reverse engineering can help with optimization

Recent work with by Alvin Cheung and Shoaib Kamil





Optimization then and now

Naïve source code



~~Optimal executable~~

Kind-of-OK executable

Domain specific problem description

SPIRAL

ATLAS

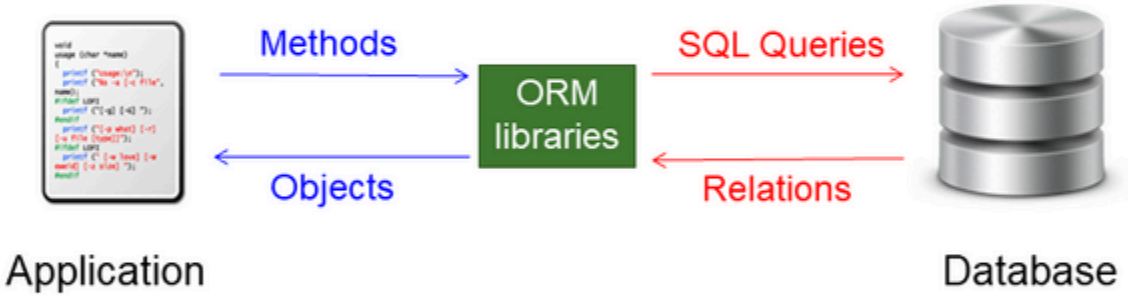
TCE

Pochoir

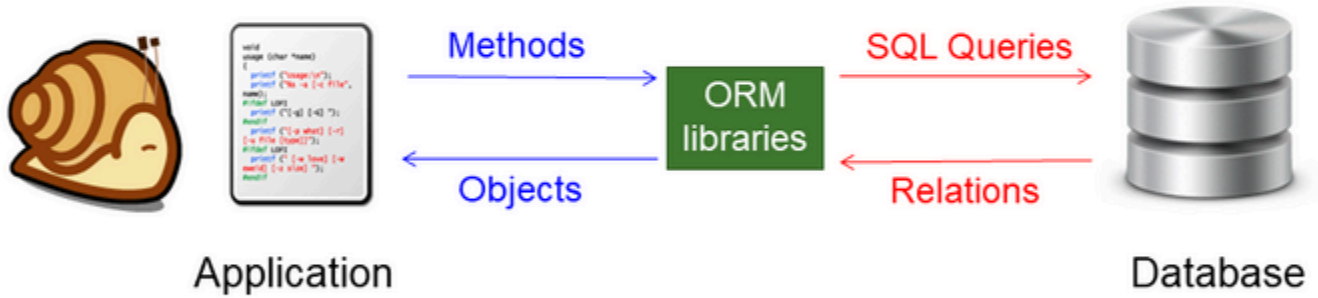
Halide

Close to optimal implementation

Java to SQL


Methods
SQL Queries
ORM libraries
Objects
Relations
Application
Database

Java to SQL


Methods
Objects
ORM
libraries
SQL Queries
Relations
Application
Database

Java to SQL

```
List getUsersWithRoles () {
  List users = User.getAllUsers();
  List roles = Role.getAllRoles();
  List results = new ArrayList();
  for (User u : users) {
       for (Role r : roles) {
         if (u.roleId == r.id)
            results.add(u); }}
  return results; }
```

SELECT * FROM user

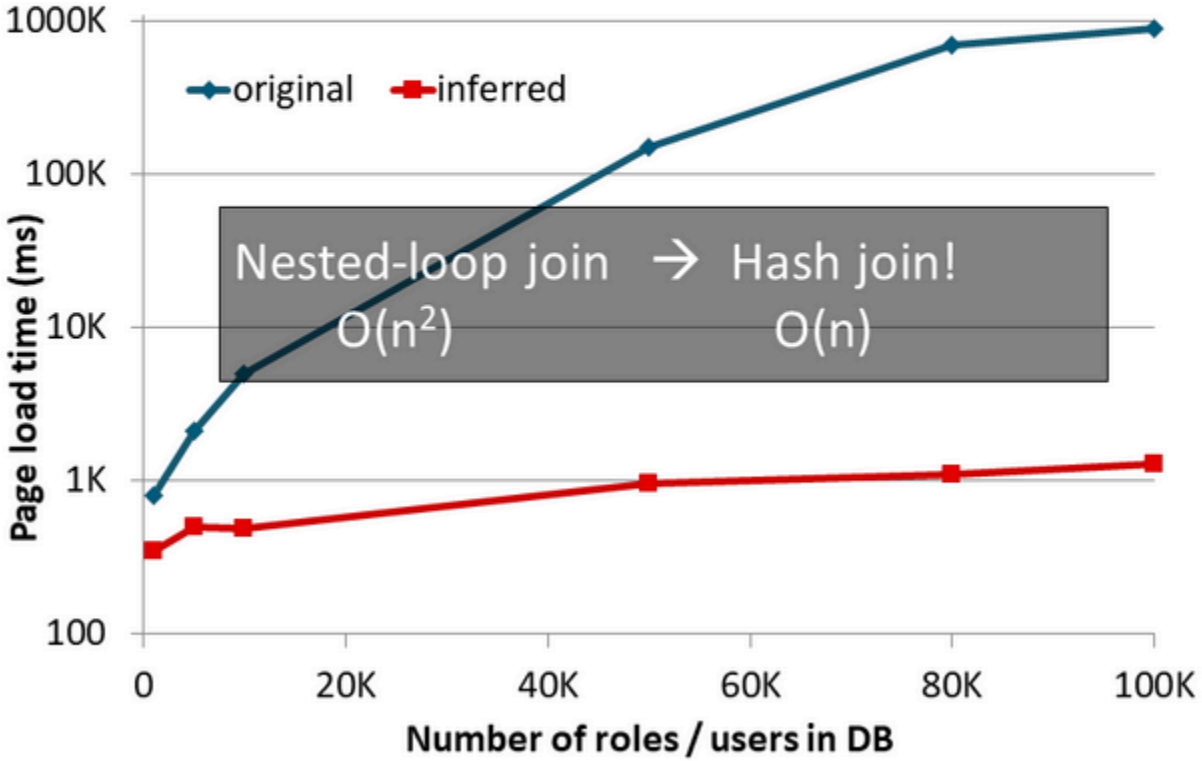
SELECT * FROM role



convert to

```
List getUsersWithRoles () {
 return executeQuery(
  "SELECT   u FROM user u, role r   WHERE    u.roleId == r.id
      ORDER BY u.roleId, r.id"; }
```

Join Query


1000K
100K
10K
1K
100
Page load time (ms)
original
inferred
Nested-loop join → Hash join!
O(n²) O(n)
0
20K
40K
60K
80K
100K
Number of roles / users in DB

Example: MultiGrid

```
DO i3 = 2, n3 - 1
DO i2 = 2, n2 - 1
DO i1 = 1, n1
r1(i1) = r(i1,i2 - 1,i3) + r(i1,i2 + 1,i3) + r(i1,i2,i3 - 1) + r(i1,i2,i3 + 1)
r2(i1) = r(i1,i2 - 1,i3 - 1) + r(i1,i2 + 1,i3 - 1) + r(i1,i2 - 1,i3 + 1) + r(i1,i2 + 1,i3 + 1)
END DO
DO i1 = 2, n1 - 1
u(i1,i2,i3) = u(i1,i2,i3) + c(0) * r(i1,i2,i3) + c(1) * (r(i1 - 1,i2,i3) + r(i1 + 1,i2,i3) + r1(i1)) + c(2) * (r2(i1) + r1(i1 - 1) + r1(i1 + 1))
END DO
END DO
END DO
```

Example: MultiGrid

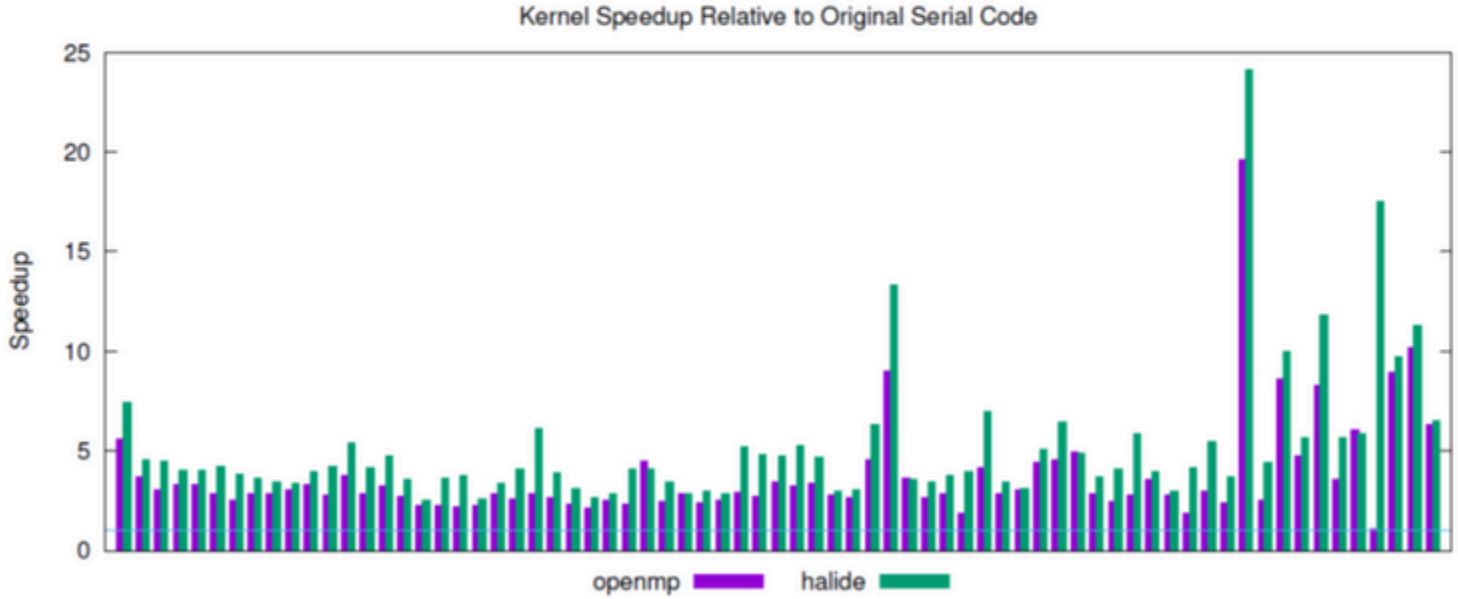
```
/*Range declarations go here */

r1_out(n1) = r(n1,n2-2,n3-1) + r(n1,n2,n3-1) + r(n1,n2-1,n3-2) + r(n1,n2-1,n3)
r2_out(n1) = r(n1,n2-2,n3-2) + r(n1,n2,n3-2) + r(n1,n2-2,n3) + r(n1,n2,n3)

u_out(i1,i2,i3) = u(i1,i2,i3) + c(0) * r(i1,i2,i3)
        +  c(1) * (r(i1 - 1,i2,i3) + r(i1 + 1,i2,i3) + r(i1,i2 - 1,i3) + r(i1,i2 + 1,i3) + r(i1,i2,i3 - 1) + r(i1,i2,i3 + 1))
        +  c(2) * ((r(i1,i2 - 1,i3 - 1) + r(i1,i2 + 1,i3 - 1) + r(i1,i2 - 1,i3 + 1) + r(i1,i2 + 1,i3 + 1))
                + (r(i1 - 1,i2 - 1,i3) + r(i1 - 1,i2 + 1,i3) + r(i1 - 1,i2,i3 - 1) + r(i1 - 1,i2,i3 + 1))
                + (r(i1 + 1,i2 - 1,i3) + r(i1 + 1,i2 + 1,i3) + r(i1 + 1,i2,i3 - 1) + r(i1 + 1,i2,i3 + 1)))

Tuple my_output(r1_out, r2_out, u_out);
```

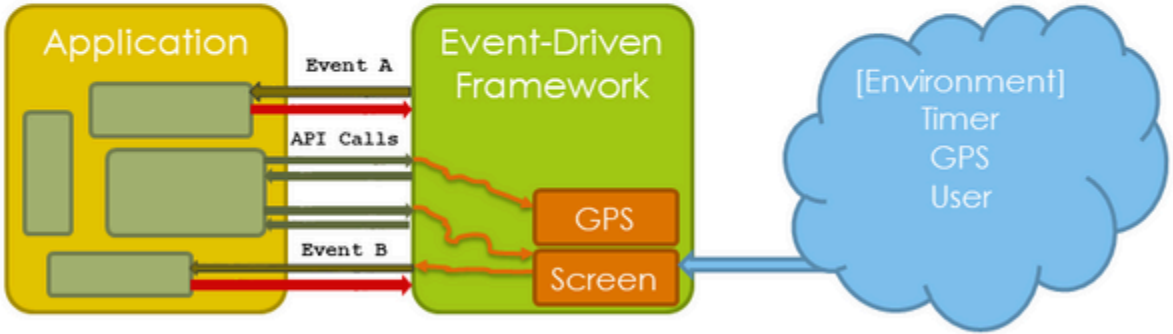
Speedups



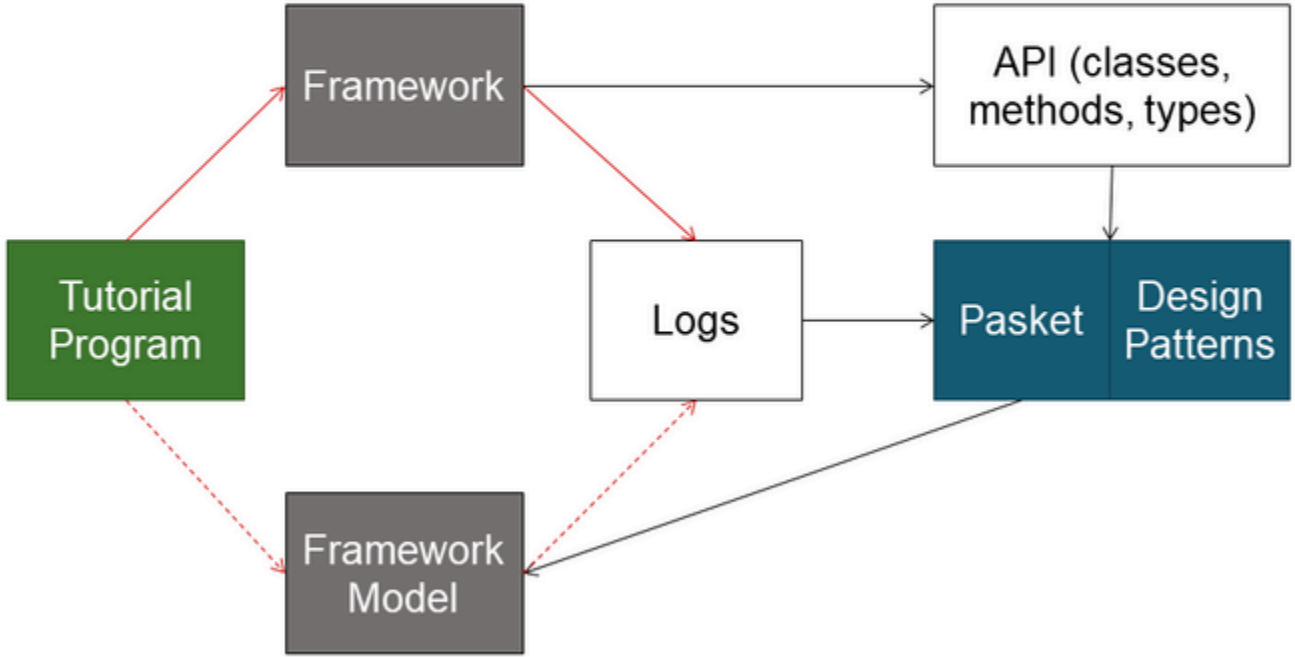
Speedups on 24 cores

Framework Models for Symbolic Execution

Pasket system by J. Jeon, X. Qiu, J. Fetter-Degges, J. S. Foster, and A. Solar-Lezama



Pasket


Framework
API (classes, methods, types)
Tutorial Program
Logs
Pasket
Design Patterns
Framework Model

JPF w(/o) Synthesized Model

```
================== search started: ...
button_demo
--------->property violated
================== error 1
gov.nasa.jpf.vm.NoUncaughtExceptionsProperty
java.lang.NoSuchMethodError:
        javax.swing.JButton.setVerticalTextPosition(I)V
  at ButtonDemo.<init>(ButtonDemo.java:63)
  at ButtonDemo.createAndShowGUI(ButtonDemo.java:131)
  ...
```

(a) With JPF's Swing model.

```
================== search started: ...
button_demo
...
================== results
no errors detected
```

(b) With PASKET's merged model.

JPF along with our synthesized model can run tutorials.

JPF's own hand-written models are insufficient.

- lack of methods: `setVerticalTextPosition`, etc.

An automated process (via Pasket) can avoid simple but nonetheless frustrating problems, like missing methods.

Automated Grading

Led by Rishabh Singh with Sumit Gulwani and myself

```
public class Program {
  public static int[] Reverse(int[] a){
    int[] b = a;
    for (int i=1; i<b.Length/2; i++){
      int temp = b[i];
      b[i] = b[b.Length-1-i];
      b[b.Length-1-i] = temp;
    }
    return b;
  }
}
```

This should be a zero

This should be a <

This should be -- instead of ++

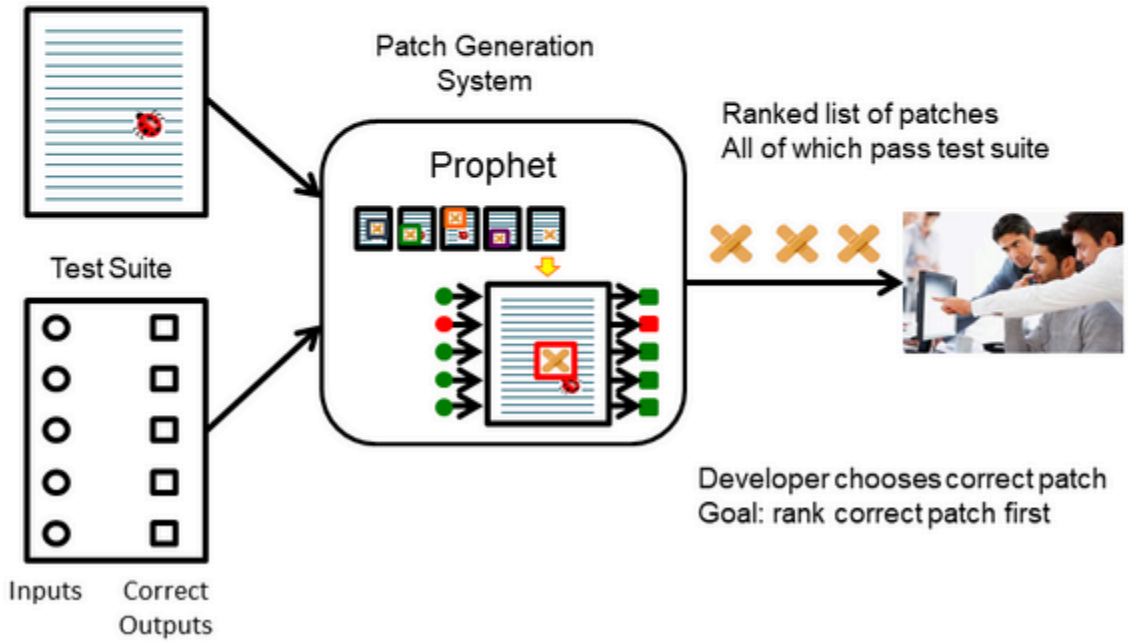
```
public class Program {
  public static int[] Puzzle(int[] b) {
    int front, back, temp;
    int[]a = b;
    front = 0;
    back = a.Length-1;
    temp = a[back];

    while (front > back)
    {
      a[back] = a[front];
      a[front] = temp;
      ++back;
      ++front;
      temp = a[back];
    }

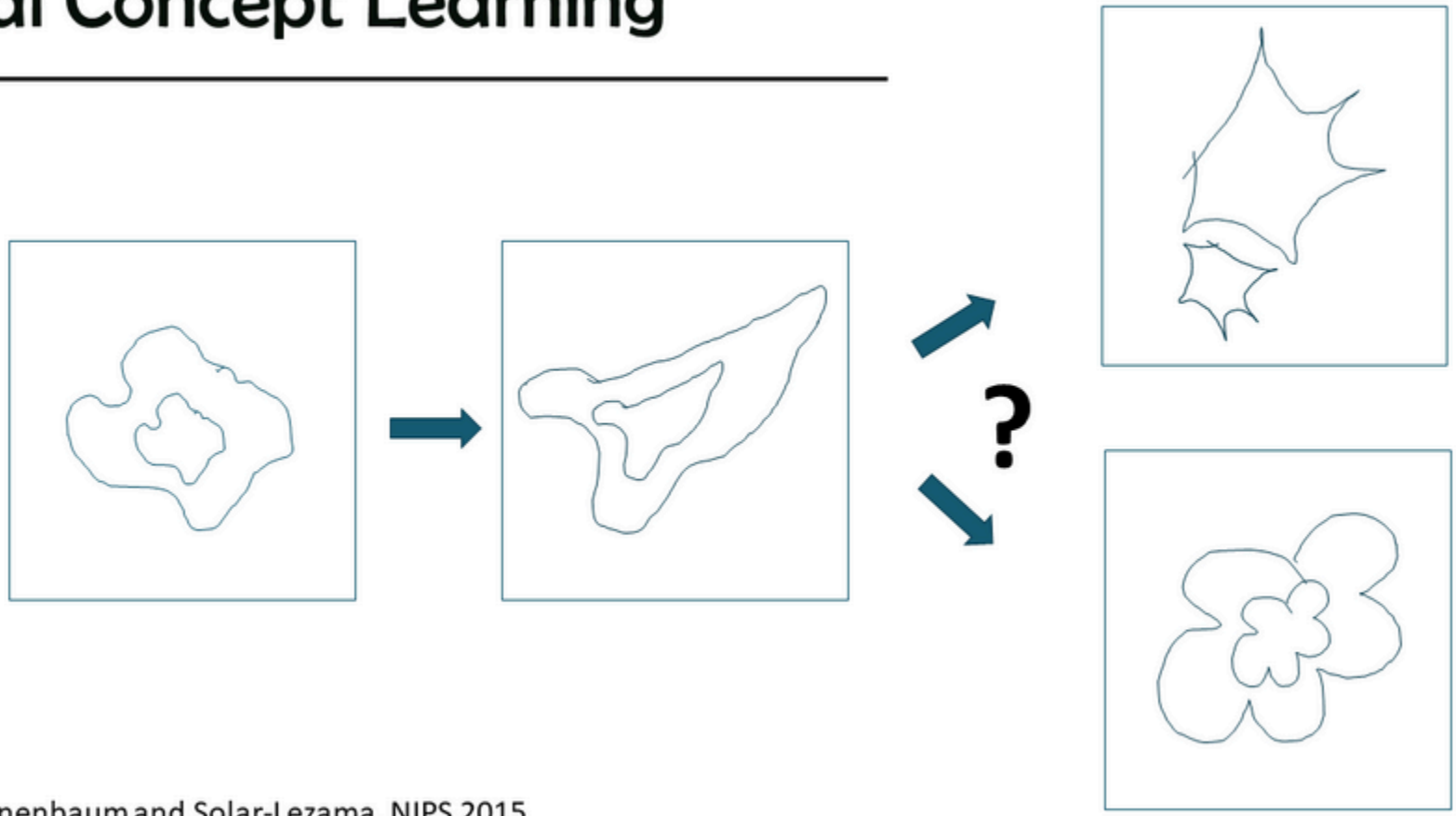
    return a;
  }
}
```

Prophet/Genesis

Project by Fan Long, Stelios Sidiroglou and Martin Rinard

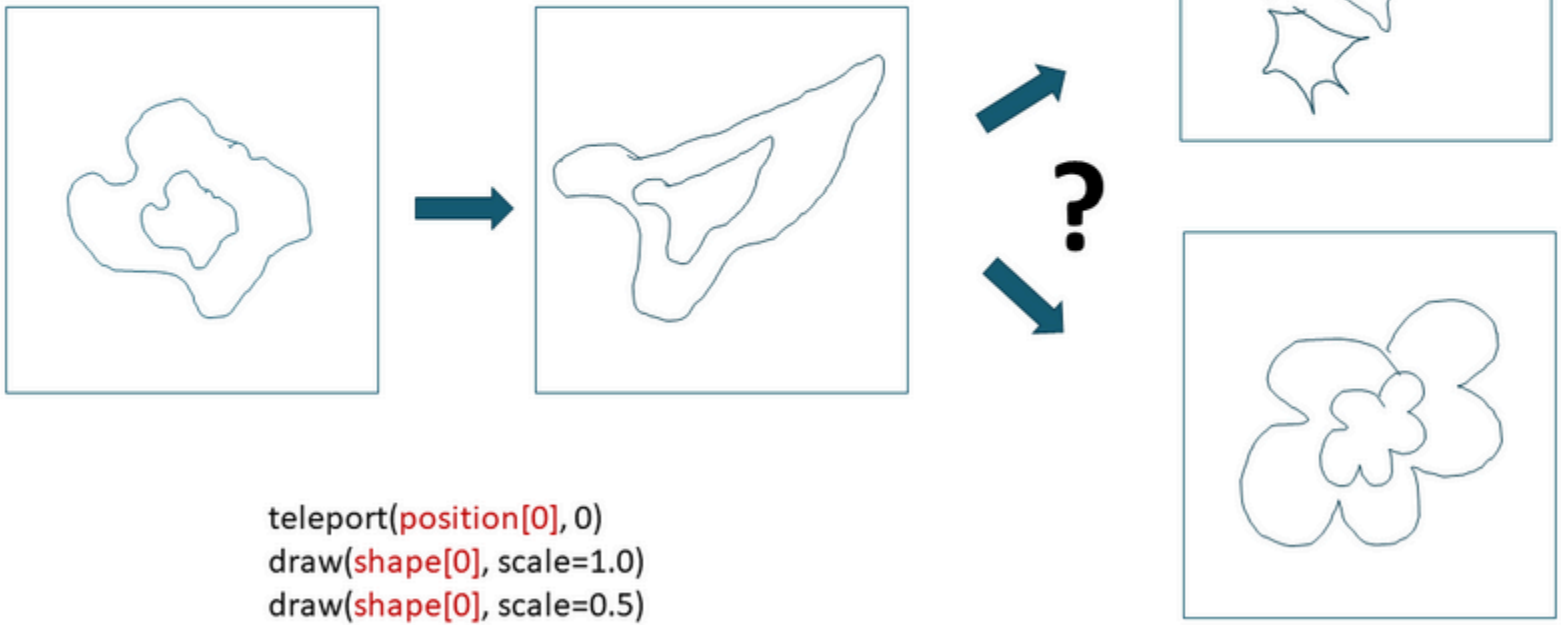


Visual Concept Learning



Ellis, Tenenbaum and Solar-Lezama, NIPS 2015

Visual Concept Learning




```
teleport(position[0], 0)
draw(shape[0], scale=1.0)
draw(shape[0], scale=0.5)
```